Github Repository Guide Agent (UID 47)

Shaik Rehna Afroz(22B3932)

February 1, 2026

Abstract

Understanding a large software repository is a major challenge for new developers. Documentation often provides high-level intent, while the actual codebase reveals architectural reality. This project presents an agentic AI system that synthesizes multiple sources of repository information— including README documentation and full file/folder structure—using the GitHub REST API and Large Language Models (LLMs). The system generates structured, onboarding-focused guided tours for GitHub repositories and exposes the analysis through an interactive Streamlit web interface. The project is developed incrementally over four weeks, emphasizing API tooling, prompt engineering, agentic reasoning, and user-facing deployment.

Contents

1 Introduction **3**

2 Project Objectives **3**

3 System Architecture **3**
3.1 High-Level Workflow . . . 3

4 Week 1: Foundations and GitHub API Tooling **4**
4.1 GitHub API Authentication . . . 4
4.2 README Retrieval . . . 4
4.3 API Error Handling Strategy . . . 4

5 Week 2: Integrating the LLM "Brain" **4**
5.1 Prompt Engineering . . . 4
5.2 Structured Summarization . . . 5

6 Prompt Engineering Strategy **5**
6.1 Role Specification . . . 5
6.2 Multi-Source Conditioning . . . 5
6.3 Output Constraints . . . 5

7 Week 3: Agentic Reasoning with Multiple Data Sources **6**
7.1 Repository Structure Traversal . . . 6
7.2 Semantic Abstraction . . . 6
7.3 Agent Prompt Design . . . 7
7.4 Agentic Behavior . . . 7

8 Week 4: Streamlit User Interface **7**
8.1 Design Principles . . . 7
8.2 Streamlit Integration . . . 7
8.3 User Experience Considerations . . . 8

9 Evaluation and Analysis **8**
9.1 Evaluation Criteria 8
9.2 Observations 8

10 DEMONSTRATION **9**

11 Security and Environment Management **10**
11.1 Secrets Handling 10
11.2 Reproducibility 10

12 Performance Considerations **11**

13 Limitations **11**

14 Engineering Best Practices **11**

15 Conclusion **11**

1 Introduction

Modern software repositories are complex, often containing thousands of files, multiple configuration layers, and heterogeneous technologies. New contributors frequently struggle to understand a repository's purpose, structure, and entry points, even when documentation is available.

This project addresses the onboarding problem by building an **agentic AI system** capable of:

- Programmatically retrieving repository metadata using the GitHub REST API
- Interpreting repository documentation and structure
- Synthesizing insights using Large Language Models
- Presenting results through an interactive web interface

Unlike simple summarization systems, the proposed agent compares *documentation intent* with *codebase reality*, enabling deeper reasoning and more accurate guidance.

2 Project Objectives

The primary objectives of the project are:

1. To build a robust Python tool for interacting with GitHub repositories using official APIs
2. To apply prompt engineering techniques for controlled and structured LLM outputs
3. To design an agent capable of reasoning over multiple data sources
4. To deliver a polished, user-friendly web application
5. To follow industry-grade software engineering practices

3 System Architecture

The system follows a modular, layered architecture:

- **Data Layer**: GitHub REST API access via PyGithub
- **Processing Layer**: Repository parsing and abstraction
- **Reasoning Layer**: Prompt-engineered LLM interaction
- **Presentation Layer**: Streamlit web interface

3.1 High-Level Workflow

1. User provides a GitHub repository in `owner/repo` format
2. README and repository structure are fetched via GitHub API
3. Raw data is transformed into compact semantic representations
4. LLM synthesizes insights using advanced prompts
5. Structured analysis is displayed to the user

4 Week 1: Foundations and GitHub API Tooling

Week 1 focuses on interacting with external web services using the GitHub REST API.

4.1 GitHub API Authentication

The system authenticates with GitHub using OAuth tokens exposed via environment variables. This avoids hardcoding secrets and follows industry-standard security practices.

Listing 1: GitHub API client initialization

```
import os
from github import Github

def get_github_client():
    token = os.getenv("GITHUB_TOKEN")
    if not token:
        raise RuntimeError("GITHUB_TOKEN␣not␣set")
    return Github(token)
```

4.2 README Retrieval

The README file is retrieved using the GitHub REST API and decoded from base64 to obtain human-readable text for downstream analysis.

Listing 2: Fetching README content

```
def fetch_readme(repo):
    readme = repo.get_readme()
    return readme.decoded_content.decode("utf-8")
```

4.3 API Error Handling Strategy

GitHub API interactions are subject to multiple failure modes. The system explicitly handles:

- **Authentication errors**: Invalid or missing OAuth tokens
- **Rate limiting**: Enforced by GitHub for excessive requests
- **Missing resources**: Repositories without README files

By distinguishing between these error classes, the system provides meaningful feedback to both developers and end users rather than failing silently.

5 Week 2: Integrating the LLM "Brain"

Week 2 introduces Large Language Models for semantic understanding.

5.1 Prompt Engineering

Explicit prompt design is used to:

- Control output structure
- Reduce hallucinations
- Target a junior developer audience

5.2 Structured Summarization

Rather than free-form summaries, the LLM is guided to produce sections such as:

- Project Purpose
- Key Features
- Technologies Used

This ensures consistency and usability.

6 Prompt Engineering Strategy

Prompt engineering is a central component of the system. Rather than issuing open-ended instructions, prompts are designed to explicitly constrain the model's role, input sources, and output format.

6.1 Role Specification

The LLM is instructed to act as a senior software engineer onboarding a new contributor. This framing improves:

- Relevance of explanations
- Technical accuracy
- Appropriate abstraction level

6.2 Multi-Source Conditioning

The prompt incorporates:

- README content (author intent)
- Repository structure summary (implementation reality)

This dual conditioning encourages the model to reconcile intent with actual code organization.

6.3 Output Constraints

The model is required to produce:

- Clearly labeled sections
- Concise bullet points
- Actionable onboarding guidance

These constraints reduce hallucination and improve consistency across runs.

7 Week 3: Agentic Reasoning with Multiple Data Sources

Week 3 represents the core contribution of the project.

7.1 Repository Structure Traversal

The full repository tree is recursively fetched using depth-limited traversal to ensure scalability.To prevent excessive API calls on large repositories, the system employs depth-limited traversal while collecting the file and folder structure.

Listing 3: Depth-limited repository traversal

```
from typing import List, Tuple

def fetch_repo_tree(repo, max_depth=3) -> List[str]:
    contents: List[Tuple[object, int]] = [
        (item, 0) for item in repo.get_contents("")
    ]
    paths = []

    while contents:
        item, depth = contents.pop(0)
        paths.append(item.path)

        if item.type == "dir" and depth < max_depth:
            for sub in repo.get_contents(item.path):
                contents.append((sub, depth + 1))

    return paths
```

7.2 Semantic Abstraction

Raw file paths are mapped into semantic categories before being passed to the LLM. This reduces prompt size while preserving structural meaning.

Listing 4: Repository structure abstraction

```
def summarize_repo_structure(paths):
    summary = {
        "source_code": [],
        "config": [],
        "docs": [],
        "tests": [],
    }

    for path in paths:
        if path.startswith(("src/", "app/")):
            summary["source_code"].append(path)
        elif path.endswith((".yml", ".json", ".toml")):
            summary["config"].append(path)
        elif "test" in path.lower():
            summary["tests"].append(path)
        elif path.lower().startswith("readme"):
            summary["docs"].append(path)

    return summary
```

7.3 Agent Prompt Design

The agent prompt explicitly instructs the LLM to:

- Compare README intent with actual structure
- Identify inconsistencies
- Recommend onboarding paths

This transforms the system from a summarizer into a reasoning agent.

The prompt explicitly constrains the model's role, inputs, and output structure, encouraging reasoning rather than surface-level summarization.

7.4 Agentic Behavior

The system exhibits agentic behavior by:

- Observing multiple sources of repository information
- Abstracting raw data into semantic representations
- Reasoning over inconsistencies between documentation and structure
- Producing recommendations rather than summaries

Unlike single-pass summarization, the agent performs synthesis across heterogeneous inputs, which is a defining characteristic of agentic AI systems.

8 Week 4: Streamlit User Interface

The final week focuses on delivering a showcasable application.

8.1 Design Principles

- Thin UI layer
- No business logic in frontend
- Clear user feedback

8.2 Streamlit Integration

Streamlit is used to:

- Accept repository input
- Trigger backend analysis
- Display formatted results

Caching is employed to improve responsiveness for repeated analyses.

The Streamlit interface serves as a thin presentation layer, delegating all reasoning to the backend agent.

Listing 5: Streamlit UI integration

```
import streamlit as st
from agent import analyze_repository

repo_name = st.text_input("GitHub␣Repository", placeholder="psf/requests")

if st.button("Analyze"):
    with st.spinner("Analyzing␣repository..."):
        analysis = analyze_repository(repo_name)
        st.markdown(analysis)
```

8.3 User Experience Considerations

The Streamlit interface is designed to:

- Minimize user input requirements
- Provide immediate feedback during analysis
- Present results in a readable, structured format

Caching mechanisms are employed to reduce repeated API calls and improve responsiveness, mirroring industry practices in production ML systems.

9 Evaluation and Analysis

The system is evaluated qualitatively across multiple public GitHub repositories with varying sizes and structures.

9.1 Evaluation Criteria

Evaluation focuses on:

- Accuracy of high-level project understanding
- Alignment between README and code structure
- Usefulness for a new developer

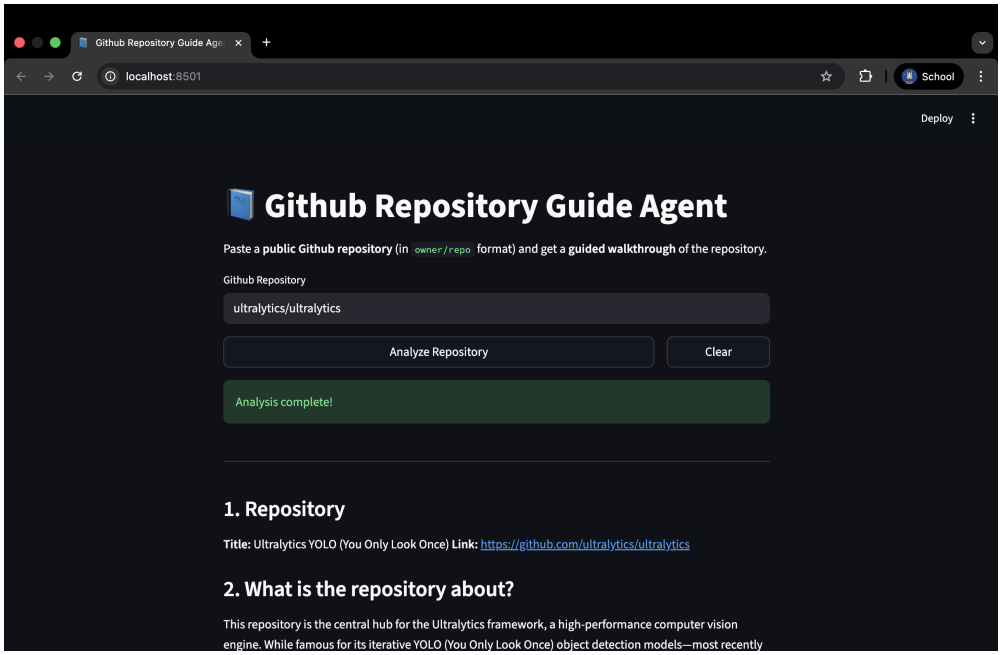
9.2 Observations

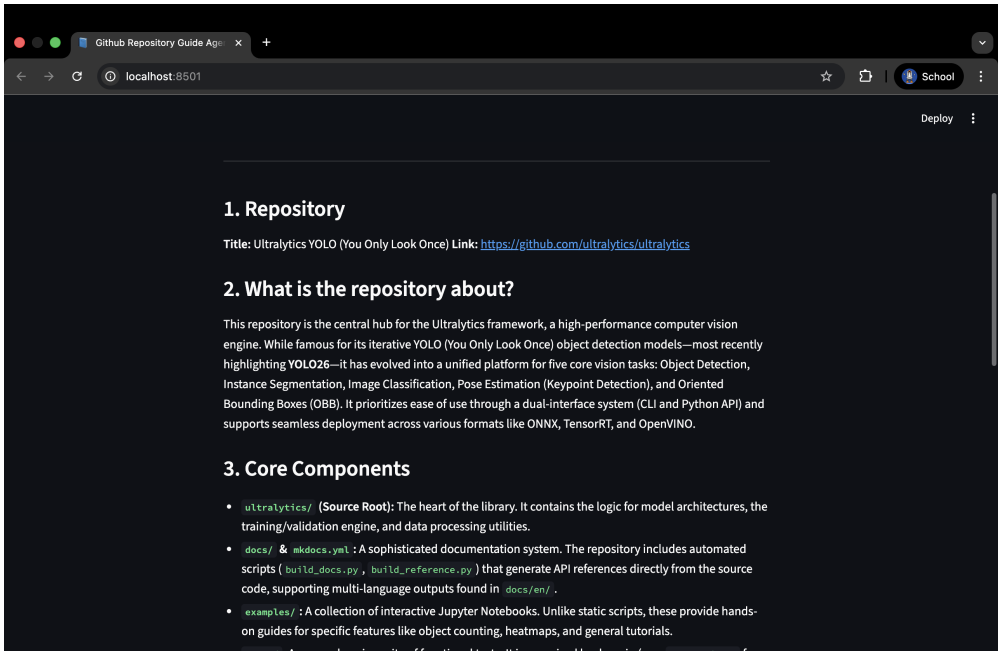
The agent performs well on repositories with:

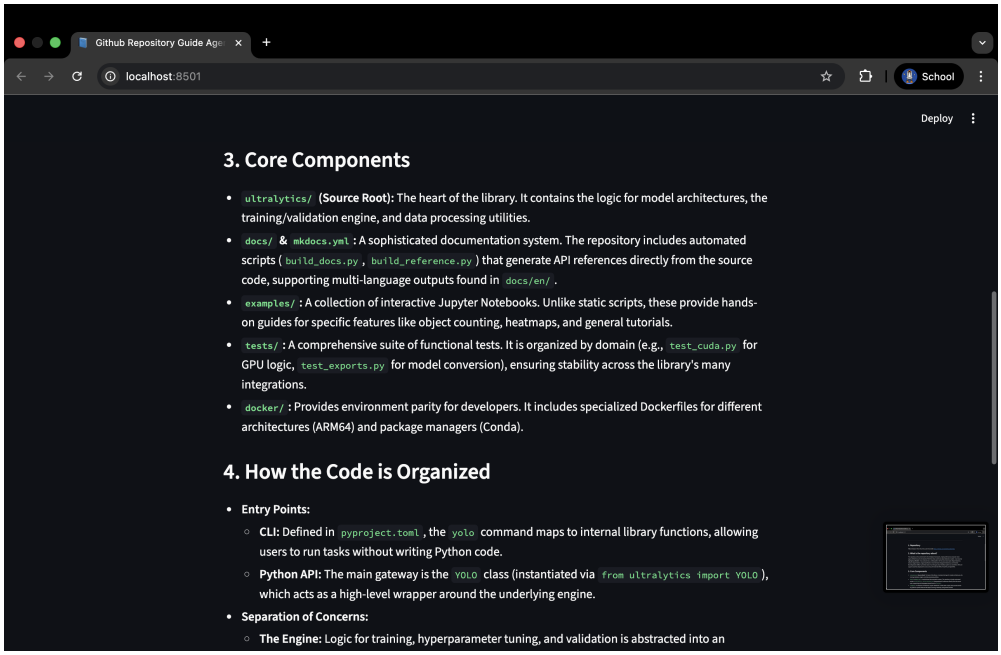
- Clear modular structure
- Meaningful README documentation

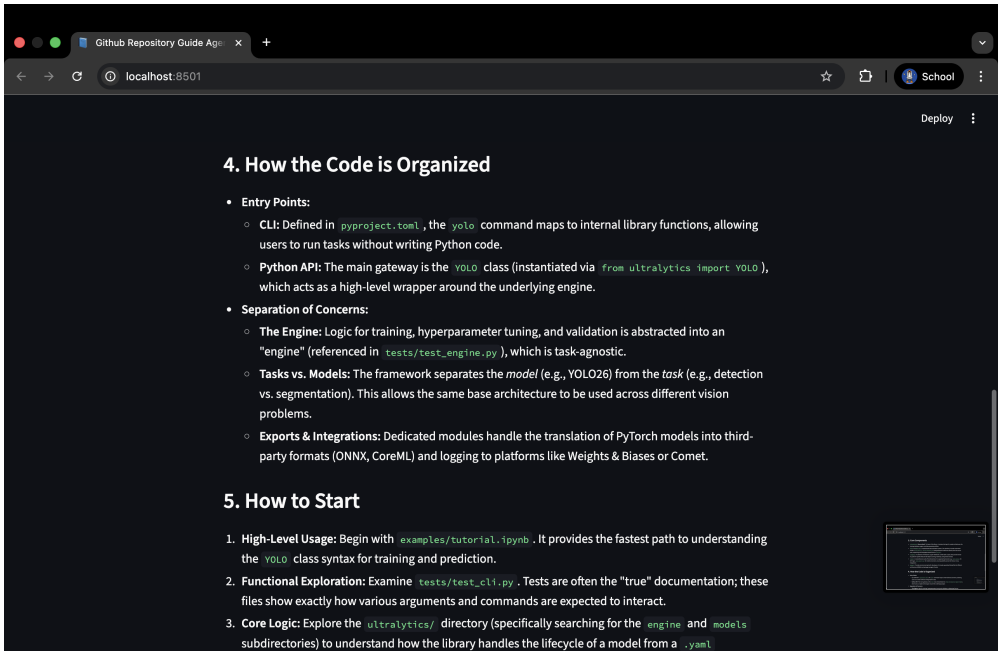
Performance degrades for repositories lacking documentation, highlighting the importance of high-quality metadata for AI-assisted analysis.

10 DEMONSTRATION









11 Security and Environment Management

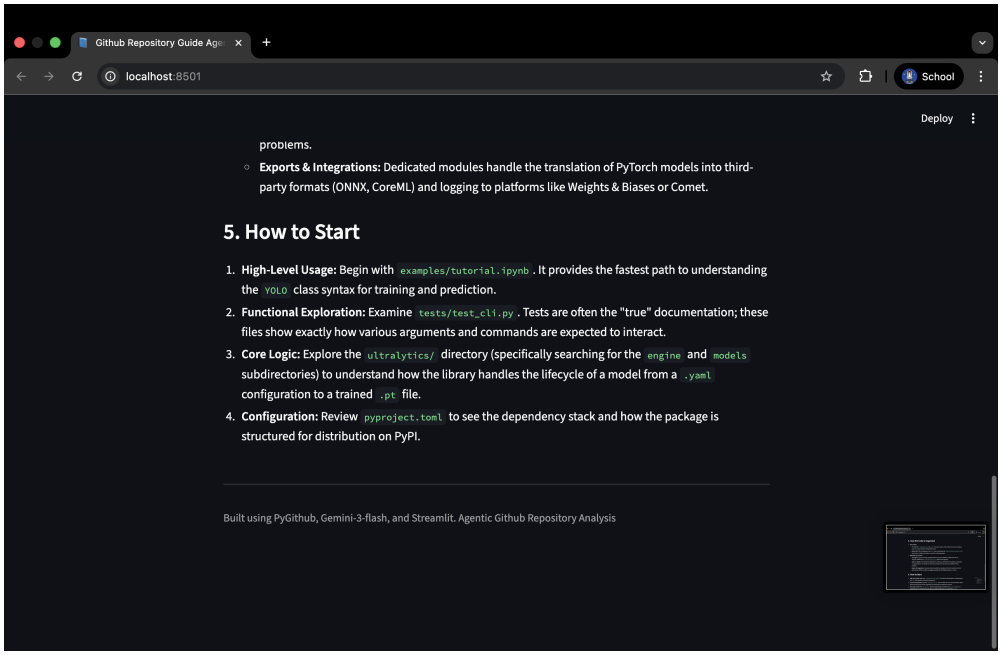
11.1 Secrets Handling

API keys are managed using environment variables:

- No hardcoded secrets
- `.env` excluded from version control
- `.env.example` provided for setup

11.2 Reproducibility

The Conda environment is specified using `environment.yml`, ensuring reproducible setups across machines.



12 Performance Considerations

Key optimizations include:

- Depth-limited repository traversal
- Structural abstraction before LLM calls
- Streamlit result caching

These strategies significantly reduce latency and API usage.

13 Limitations

- Large repositories may still incur higher latency
- Analysis quality depends on README quality
- LLM responses may vary due to probabilistic nature

14 Engineering Best Practices

The project adheres to several industry-standard best practices:

- Modular code organization
- Secure handling of secrets via environment variables
- Reproducible environments using Conda
- Clear separation of backend logic and frontend presentation

These practices ensure maintainability, scalability, and ease of collaboration.

15 Conclusion

This project demonstrates the end-to-end development of an agentic AI system that combines API tooling, structured reasoning, and interactive deployment. By synthesizing repository documentation and structure, the system provides meaningful guidance for developers and reflects industry-grade engineering practices.